Investigation Report: Table 1 (Five-System Comparison) Data Source Cannot Be Reproduced

CI debugging session — `ci_postprocess.yml` / `exp1_ablation`

August 1, 2026

Executive Summary

`generate_tables.py`'s `gen_five_system()` produces Table 1 (`tab:five_systems_full`, the Extrapolation Error vs. Interpolation R^2 comparison across five systems). When no JSON file with a `"five_system"` or `"system_comparison"` key is found, it silently falls back to a hardcoded constant, `FIVE_SYSTEM_PAPER_ROWS`.

This investigation set out to locate the real experiment that should feed this table and wire it in. It instead found that **no data source currently present in the repository reproduces the numbers in `FIVE_SYSTEM_PAPER_ROWS`** — not for any of the five rows, and not even in cases where sample sizes coincidentally match. Two structurally different experiment pipelines were found that plausibly contributed *some* of the fallback's rows, using two incompatible metric definitions, on two different sets of test problems. Neither, on close inspection, reproduces the published numbers exactly.

The practical conclusion: **Table 1 is not currently reproducible from any live pipeline in this repository.** This is a data-provenance problem, not a wiring bug, and should be resolved as a decision (what should this table measure, and from what source) before `generate_tables.py` is patched to point at anything.

1 Background: How the Fallback Was Discovered

The CI job `ci_postprocess.yml` runs `scripts/generate_tables.py` for `experiment=exp1_ablation`. Without `--allow-fallback`, the script's missing-JSON audit finds no file with a `"five_system"` / `"system_comparison"` key anywhere under its search candidates and exits non-zero, hard-blocking the CI step (no `|| true`, and the step runs under `set -euo pipefail`).

`generate_tables.py` searches, in order, every `*.json` file (not just the newest by mtime) under each of:

- `ablation/exp1_ablation/*.json` *(unconfirmed as the right directory)*
- `comparison_results/five_system/*.json`
- `comparison_results/system_comparison/*.json`
- `five_system/*.json`
- `system_comparison/*.json`

- `five_system*.json` / `system_comparison*.json` (repo root level)

None contain the required key. This is the documented, intentional fallback behavior (`Issue 4` in the script's own comments) — the question this report investigates is *where the real data should come from instead.*

2 Data Sources Examined

2.1 `exp2` / Feynman 10-domain comparison (candidate A)

Producer: `run_comparative_suite_benchmark_v2.py --protocol all_domains`, invoked per-domain by the `exp2` step in `run_all.sh` across 10 domains (mechanics, thermodynamics, electromagnetism, fluid_dynamics, optics, quantum, chemistry, biology, mathematics, economics).

Methods run (6, from `METHOD_REGISTRY`):

#	Class	JSON result key
1	`PureLLMBaselineMethod`	`PureLLM Baseline (core)`
2	`ImprovedNNMethod`	`ImprovedNN (core)`
3	`HybridDeFiMethod`	`EnhancedHybridSystemDeFi (core)`
4	`HybridAllDomainsMethod`	`HybridSystemLLMNN all-domains (core)`
5	`SymbolicEngineMethod`	`SymbolicEngineWithLLM (tools)`
6	`HybridSystemV50_2Method`	`HybridDiscoverySystem v50_2 (tools)`

Output files:

- `comparison_results/feynman-tests/exp2_multi/protocol_core_noiseless_*.json` — 10 timestamped shards, in-distribution results (30 unique (domain, test) records after dedup by latest timestamp).
- `comparison_results/feynman-tests/exp2_extrap/benchmark_results_extrap.json` — 180 flat records (30 equations × 6 methods), produced by the separate `exp2_feynman_extrap` step (`--extrap --extrap-train-frac 0.8 --extrap-multiplier 2.0`).
- Flat `benchmark_results.json` (non-extrap): **never produced** — the export is wrapped in a bare `try/except Exception` in the producer script and silently failed; no file exists on disk. Not fatal, since the nested `protocol_core_noiseless_*.json` files carry the same data.

Finding: `compute_extrap_r2_far()` — the function that produces `extrap_r2_far` / `extrap_error_pct` — only evaluates methods whose `formula` field is a re-evaluable symbolic expression string. Any method returning an NN architecture tag (`"ImprovedNN(...)"`), an `"[NN fallback"` tag, or `"N/A"` is skipped and receives `None` for all three extrapolation fields, per the function's own docstring: *"Only SymbolicEngineWithLLM and HybridDiscoverySystem v50_2 produce formula strings and yield non-null values."* This is a structural property of the code, not a data gap: **Neural Network can never have extrapolation data from this pipeline, regardless of how many times it is re-run.**

Real per-method statistics (exp2 / exp2_extrap)

Real method	Paper row	train_r2_mean	train n	extrap n
PureLLM Baseline (core)	Pure LLM	1.000	27/30	4
ImprovedNN (core)	Neural Network	0.998	30/30	**0**
SymbolicEngineWithLLM (tools)	System 2 Symbolic	1.000	30/30	18
HybridSystemLLMNN all-domains (core)	System 3 LLM+Fallback	1.000	30/30	**0**
HybridDiscoverySystem v50_2 (tools)	Hybrid v50_2	0.998	30/30	14

`EnhancedHybridSystemDeFi (core)` (method #3) is excluded from the above mapping: it is DeFi-domain-scoped (its module lives under `hybrid_defi_system/`), uses an `'ensemble'` decision strategy distinct from method #4's explicit LLM-with-NN-fallback pattern, and does not correspond to any of the five paper row names.

extrap median % is computed from the pre-existing `extrap_error_pct` field in `benchmark_results_extrap` (defined as $\mathrm{RMSE}_{\mathrm{far}}/\mathrm{RMSE}_{\mathrm{train}} \times 100$ in `calculate_extrapolation_error()`); no metric was invented for this report. Raw (unclipped) means are dominated by catastrophic single-equation blowups — e.g. Pure LLM's raw mean is $\approx 4.6 \times 10^9$% from one degenerate formula — so only the median is reported here as representative.

2.2 `comparison_results/extrapolation/` (near-duplicate of A)

A second, separately-dated (`2026-07-16` vs. exp2_extrap's `2026-07-14`) `benchmark_results_extrap.json` exists at `comparison_results/extrapolation/`, produced by the `all_domains_extrap_v4` step in `run_all.sh`. On inspection this is effectively a re-run of the same experiment: identical method set (6 methods), identical record count (180), and identical statistics — e.g. Pure LLM's median error % matches exp2_extrap's to the decimal (2157.2).

Finding: This directory is not an independent data source; it is a duplicate run of candidate A and does not resolve any discrepancy.

2.3 `extrapolation_73cases_enhanced.json` (candidate B)

Producer: `run_hybrid_system_benchmark.py`, an entirely separate harness from `run_comparative_suite_benchm`

Domain: DeFi/finance formulas (Value at Risk, options pricing, AMM arbitrage, impermanent loss, etc.) — **not** the Feynman physics domains used by exp2.

Methods run (3, hardcoded): `pure_llm`, `neural_network`, `hybrid` — a fundamentally different, coarser method set than exp2's six.

Metrics: `test_r2`, `extrapolation_gap`, `stability_score` — computed directly per-record, not via the RMSE-ratio `extrap_error_pct` used in candidate A. These are **not the same metric** even where both produce a percentage-like number.

Case pool: `TOTAL_EXTRAP_CASES = 73`, of which 6 are excluded as `_INTRACTABLE_CASES` (piecewise/degenerate formulas where the test split collapses), leaving 67 "standard" cases.

Why candidate B was suspected

The paper's fallback shows `Neural Network: n=13` and `Hybrid v50_2: n=14` — small counts inconsistent with exp2's 30-equation set, but plausibly consistent with a filtered subset of a 67-case

DeFi pool. Candidate B's method names (`pure_llm`, `neural_network`, `hybrid`) also linguistically match three of the five paper row names far more directly than exp2's verbose class-derived names.

Why candidate B was ruled out

Counting non-`None` `test_r2` values on the 67-case standard subset, exactly as `run_hybrid_system_benchmark.py`'s own `_collect()` helper does:

Method	n (standard subset, non-None test_r2)
pure_llm	67
neural_network	67
hybrid	67

All three methods return 67, not 13 or 14. The report-generation function's own `_robust_stats()` was also checked directly for any additional filtering (a quality threshold, a clip-based exclusion, etc.) beyond NaN removal that might explain a further reduction from 67 down to 13–14:

```
valid = [s for s in scores if not np.isnan(s)]
...
n = len(valid)
```

No such filtering exists — `n` is simply the count of non-NaN scores.

Finding: Candidate B does not reproduce the paper's `n=13`/`n=14` counts under any filtering rule found in its own source code.

3 Comparison: Real Data vs. Published Fallback

Row	Paper (fallback)		Real (exp2, candidate A)		
	n	extrap %	n	extrap median %	
Pure LLM	0	—	4	2157.2	*paper has no dat*
Neural Network	13	86.7	**0**	—	*structurally impossible fro*
System 2 Symbolic	0	—	18	112.2	*paper has no dat*
System 3 LLM+Fallback	0	—	**0**	—	*agrees (both empty), different r*
Hybrid v50_2	14	0.0	14	125.7	*n matches; magnitude off by 10^2*

Finding: Even the single closest match (Hybrid v50_2, $n = 14$ in both) shows a two-to-three-order-of-magnitude disagreement in the reported extrapolation error (0.0% published vs. ≈125.7% recomputed). A published value of exactly 0.0% implies perfect extrapolation; the real recomputed value indicates far-region error comparable to or exceeding training error. This is not attributable to rounding, a different random seed, or a slightly different equation subset — it indicates a different metric definition, a different (older or since-modified) code path, or numbers that were not machine-generated from the current pipeline at all.

4 Root Cause Assessment

The evidence supports the following conclusion with moderate-to-high confidence:

1. **The five-row table was assembled from at least two structurally incompatible sources**, not from one coherent experiment run: some subset resembling a DeFi/3-method/R^2-gap harness (candidate B in spirit, though not an exact match), and the remaining rows left entirely unpopulated (`"---"`) because no source was ever wired up for them.

2. **The paper's own fallback already concedes this**: three of five rows (Pure LLM, System 2 Symbolic, System 3 LLM+Fallback) are `n=0`, `"---"` placeholders in `FIVE_SYSTEM_PAPER_ROWS` itself. This is not a regression introduced by this investigation — the hardcoded "paper-verified" constant was already incomplete for 60% of its extrapolation columns.

3. **No currently-existing script, if re-run today, would reproduce the published Neural Network or Hybrid v50_2 numbers exactly**, despite exhaustive search across every JSON output directory referenced by `run_all.sh` for extrapolation-shaped data (`exp2_multi`, `exp2_extrap`, `comparison_results/extrapolation/`, `hybrid_pysr/defi/`, and the standard `_FIVE_SYSTEM_CANDIDATES` search paths).

Open question: Where did the published `n=13` (Neural Network) / `n=14` (Hybrid v50_2) / `0.0%` (Hybrid v50_2 error) numbers originate? Candidates not yet checked: an older git-history version of `run_hybrid_system_benchmark.py` with a different `_INTRACTABLE_CASES` set or a different clip/threshold rule; a manually curated subset; or a run whose output file has since been deleted or was never committed. `git log -p` on `run_hybrid_system_benchmark.py` and `generate_tables.py` (specifically the commit that introduced `FIVE_SYSTEM_PAPER_ROWS`) would be the next step if this is pursued further.

Open question: Is `train_r2_mean` in the paper's intended table meant to be full-dataset R^2 (as computed by the noiseless exp2 run) or training-split-only R^2 (as computed internally by the `--extrap` run before evaluating the far region)? This was never resolved and affects which of exp2's two output files is the correct source even for the columns that *do* have real data.

5 Recommendation

Do not patch `_load_five_system_data()` / `_rows_from_data()` to point at `exp2/exp2_extrap` as a drop-in fallback replacement. Doing so would silently swap one set of unverified numbers for a different, differently-computed set, without resolving why they disagree, and would misrepresent Table 1 as "regenerated from real data" when the real data measures a different metric on different problems for at least one row (Hybrid v50_2) and cannot populate two rows at all (Neural Network, System 3 LLM+Fallback) under the current codebase.

Suggested path forward, in order:

1. Determine, from the paper text or its authors, what Table 1 is actually meant to measure and over what problem set (Feynman-only, DeFi-only, or a deliberate cross-domain comparison).

2. If cross-domain, consider whether Table 1 should be split into two tables with clearly different scope, rather than one spliced table.

3. Locate or re-derive the true source of the published Neural Network/Hybrid v50_2 numbers (see open questions above) before treating any current script's output as their replacement.

4. Only once (1)–(3) are resolved, patch `generate_tables.py` to read from the confirmed real source, and update `FIVE_SYSTEM_PAPER_ROWS`'s status from "paper-verified fallback" to either "superseded by live data" or "confirmed historical value, source: …".